Hydroficient IoT Cyber Defense
Project 4: Secure the Pipeline & Measure the Cost

Step 1: Understanding Encryption

Why Encryption Matters

There is no way to monitor if someone's actually eavesdropping on the pipeline right now? Passive eavesdropping is invisible. If someone's watching, we'd never know.

What an Attacker Can Learn

From unencrypted MQTT traffic, an attacker could discover:

Information	How They Get It	Why It's Dangerous
Sensor locations	Topic names like grandmarina/pool_spa/	Maps your infrastructure
Occupancy patterns	Flow rate spikes = rooms in use	Privacy violation, burglary planning
System vulnerabilities	Normal pressure ranges	Knows what "abnormal" looks like
Message timing	When sensors report	Can plan attacks for quiet periods

This is called **reconnaissance** — gathering information about a target before launching a real attack. The terrifying part? Reconnaissance is almost impossible to detect. The attacker isn't sending anything. They're just receiving. Your system has no idea they're there.

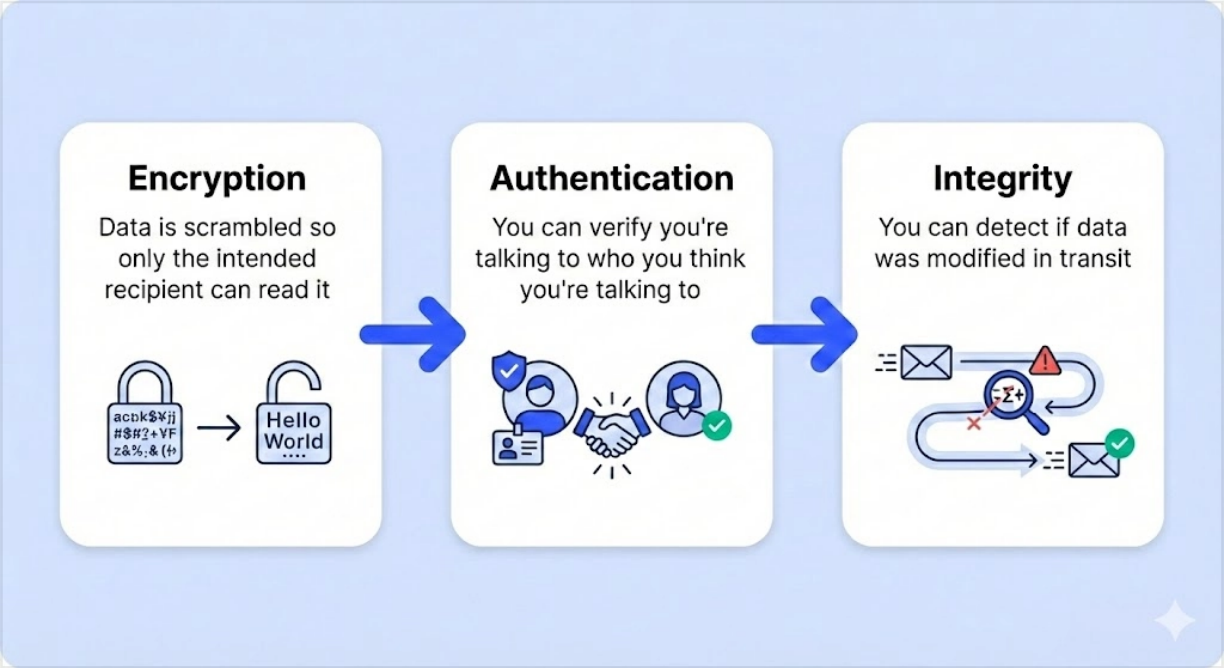
Real-World Incidents

- **Oldsmar, Florida (2021):** In February 2021, attackers accessed a Florida water treatment plant and tried to increase sodium hydroxide (lye) to dangerous levels — 100 times higher than normal. An alert operator caught the change and reversed it. While the attackers got in through remote access software (not MQTT), the incident showed how vulnerable water infrastructure can be. If communications had been unencrypted, reconnaissance would have been trivially easy.

- **The Mirai Botnet (2016):** In 2016, the Mirai botnet compromised over 600,000 IoT devices — security cameras, routers, DVRs — largely because they used default passwords and unencrypted communications. The resulting DDoS attack took down major websites including Twitter, Netflix, and Reddit.

Understanding TLS
TLS — Transport Layer Security. It's the same technology that puts the 'S' in HTTPS. Every time you see that padlock in your browser, TLS is working behind the scenes. TLS has nothing to do with HTTP specifically. It sits between the application and the network layer — which means it works for *anything*. Web browsers, instant messaging, VPNs, email... and yes, MQTT. TLS is a cryptographic protocol that doesn't care what's riding on top of it. Our sensors, our broker, our dashboard — they can all use TLS the same way your bank's website does.

TLS provides three critical protections:



Here's what that looks like for your pipeline:

Without TLS	With TLS
Messages readable by anyone on the network	Messages encrypted — only sender and receiver can read them
No verification of who you're talking to	Certificates prove the broker is legitimate
Attacker can easily eavesdrop	Eavesdropper sees only encrypted garbage

MQTT vs MQTTS
Just like HTTP has HTTPS, MQTT has MQTTS. 1883 is insecure MQTT — what you've been using. 8883 is MQTTS — MQTT with TLS. The 'S' stands for 'Secure.' When you see port 8883, you know encryption is expected.

💡 **What's a port?** Think of an IP address as a building's street address. A port is the door number inside that building. A single server can run many services — each one listens on a different port. Port 80 is for web traffic, port 443 is for secure web traffic, port 1883 is for MQTT, and port 8883 is for MQTT with TLS. When your sensor connects to the broker, it needs both the address (where) and the port (which door).

Switching ports tells the broker to expect TLS, but you still need to set up certificates, configure the broker, update your Python code

Understanding Certificates

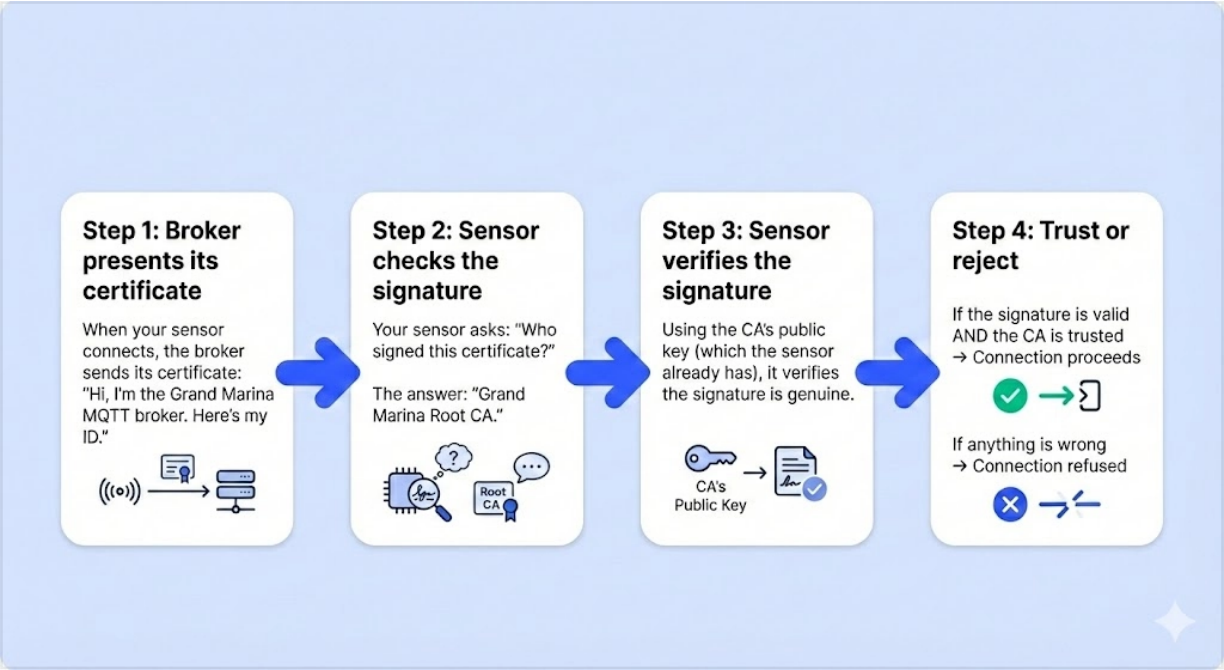
Why Encryption Alone Isn't Enough
An attacker sets up a fake broker. Your sensor connects to it, thinking it's real. The fake broker decrypts your data, reads everything, then passes it to the real broker. From your perspective? Everything looks normal. Your data is "encrypted" — just to the wrong person.
This is called a **Person-in-the-Middle (PITM) attack**. The encryption works perfectly... it's just protecting the conversation between you and the attacker.
The solution: Your sensor needs to verify — "Is this really the Grand Marina broker, or an imposter?"
That's what **digital certificates** do. They prove identity.

What is a Digital Certificate?
Think of a digital certificate like a **passport for a server**.
When you travel internationally, border agents don't just take your word for who you are. They check your passport — which has your photo, your information, and critically, it's issued by a government they trust.
A certificate works the same way. It contains:

Field	What It Is	Example
Subject	Who this certificate belongs to	localhost or broker.grandmarina.com
Issuer	Who vouches for this identity	Grand Marina Root CA
Public Key	Used for encryption	(A long string of characters)
Valid From/To	When the certificate is valid	2024-01-01 to 2025-01-01

Signature	Cryptographic proof the issuer approved this	(Digital signature)

The **signature** is the critical part. Anyone can claim to be the Grand Marina broker. The signature proves it — and it's impossible to forge without the issuer's private key. Before any data is exchanged, Your sensor won't send a single pressure reading until it's verified who it's talking to.



Certificate Authority (CA): The Passport Office

A **Certificate Authority (CA)** is the trusted entity that signs certificates.

Think of it as the passport office of the digital world. The passport office verifies your identity, then issues a passport with its official stamp. Anyone who trusts the passport office trusts the passports they issue.

For public websites, well-known CAs like Let's Encrypt, DigiCert, and Sectigo sign certificates. Your browser comes pre-installed with about 150 trusted CAs — that's why when you visit your bank, your browser already knows to trust the certificate.

For Grand Marina? We'll create our **own CA**. We're becoming our own passport office for the hotel's water monitoring system.

Why Create Our Own CA?

For internal systems, this is standard practice:

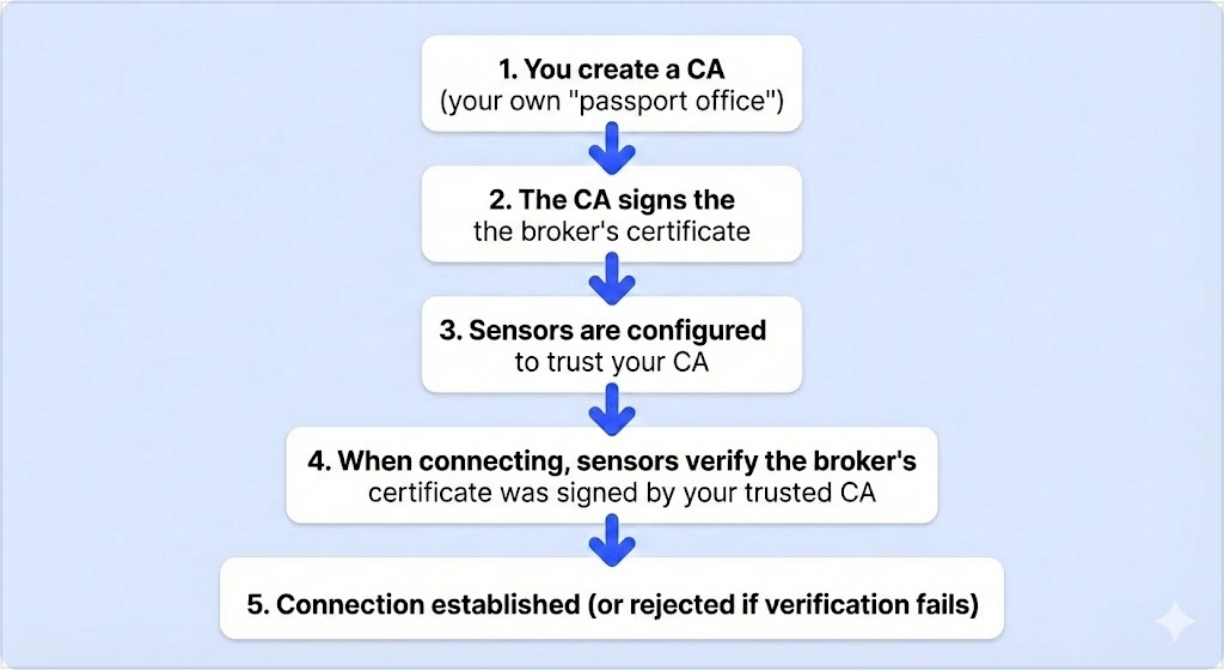
Public CA	Private CA (What We'll Use)
Trusted by browsers globally	Trusted only by your systems
Requires domain verification	You control everything

Free or paid	Free
Good for public websites	Good for internal systems

We don't need the whole world to trust our broker — just our sensors. This is how most industrial IoT systems work.

Important: Certificates are meant to be shared — that's how trust is verified. But the private key stays locked down. If someone gets your private key, they can impersonate your broker. Game over.

The Chain of Trust:



Your sensors trust your CA. Your CA vouched for the broker. Therefore, your sensors trust the broker.

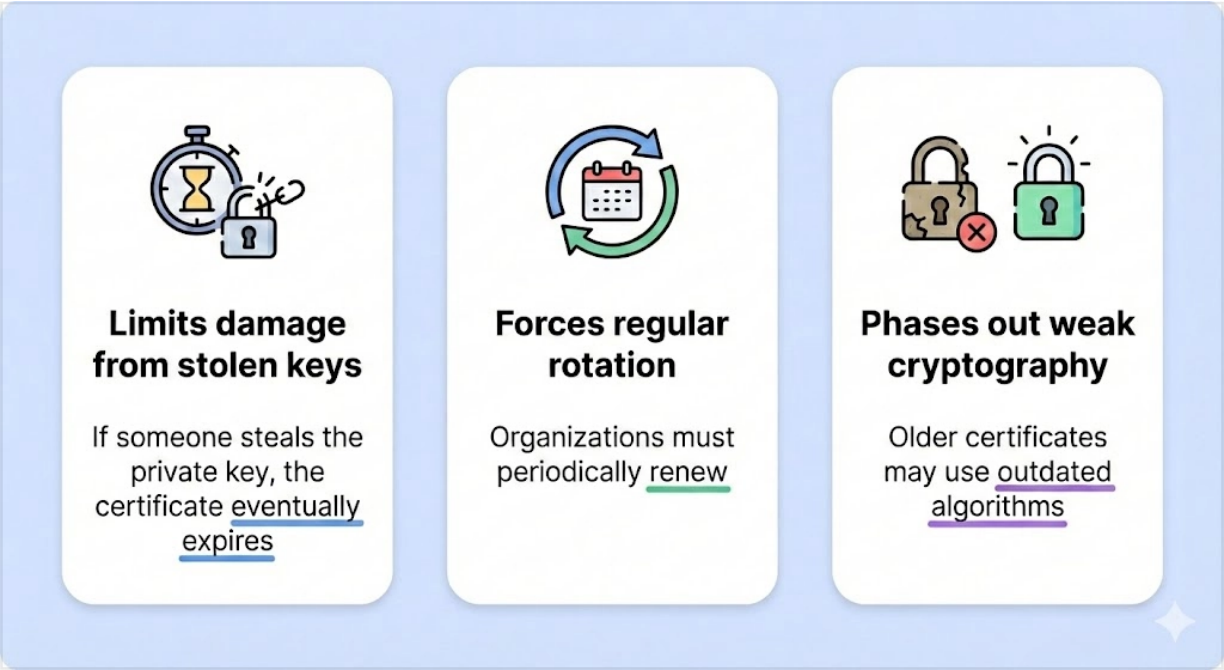
One-Way vs Mutual TLS

There are two flavors of TLS:

One-Way TLS (This Week)	Mutual TLS / mTLS (Week 5)
• Broker has a certificate • Clients verify the broker's identity • Clients don't need their own certificates • This is how most websites work — you verify the bank is real, but the bank doesn't verify you via certificate	• Both broker and the clients have certificates • Everyone verifies everyone • "Only sensors with valid certificates can connect" This week: secure the channel. Week 5: secure who's allowed in.

Why Certificates Expire

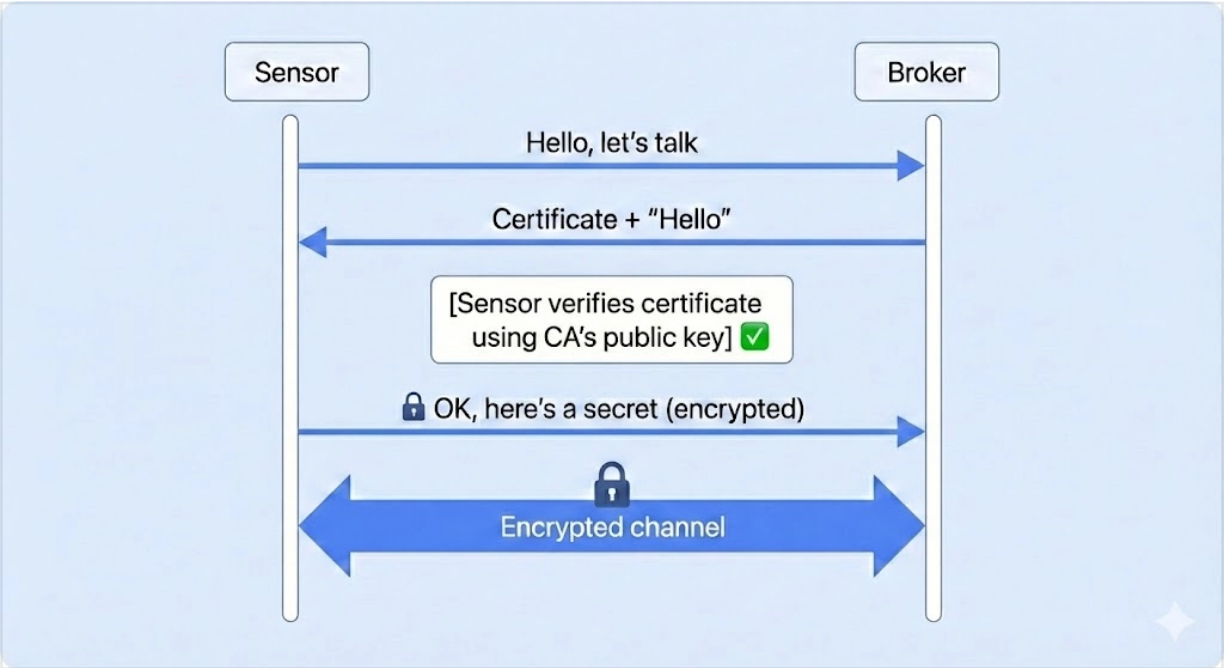
Expiration is a security feature, not a bug:



If you try to use an expired certificate? Connection fails. Immediately. No warnings — just refused.

💡 **Industry Trend**
Certificate lifetimes are getting shorter. The industry is moving toward 47-day certificates (down from 1-2 years). For our learning environment, we'll use 1-year certificates for the server and 10-year for the CA.

The TLS Handshake (Simplified)
Here's what happens when your sensor connects to the broker:



That double line at the bottom? That's the encrypted channel. After the handshake, ALL your MQTT messages flow through it. The handshake takes milliseconds, but establishes a secure tunnel for the entire session. Trust first, then communicate.

Encryption Scavenger Hunt

What You'll Do
Examine certificates from three websites and record what you find. You'll see real CAs, real expiration dates, and real certificate chains. Watch this quick video to understand the process.

Follow the Steps Below

- **Open Your Browser:** Use Chrome, Firefox, or Edge. Safari works too, but the steps are slightly different.
- **Visit a Website with HTTPS:** Start with your bank's website or https://github.com. Make sure the URL starts with https://.
- **View the Certificate**

In Chrome/Edge:	**In Firefox:**
1. Click the padlock icon in the address bar 2. Click "Connection is secure" 3. Click "Certificate is valid" 4. A certificate viewer will open	1. Click the padlock icon in the address bar 2. Click the arrow next to "Connection secure" 3. Click "More Information" 4. Click "View Certificate"

- **Record What You Find:** Create a new Google Doc. For each website, find

and record. Complete this for **three different websites.**

Field	What to Find
Subject (Issued To)	The domain name or organization
Issuer (Issued By)	The Certificate Authority
Valid From	When the certificate was created
Valid To	When it expires

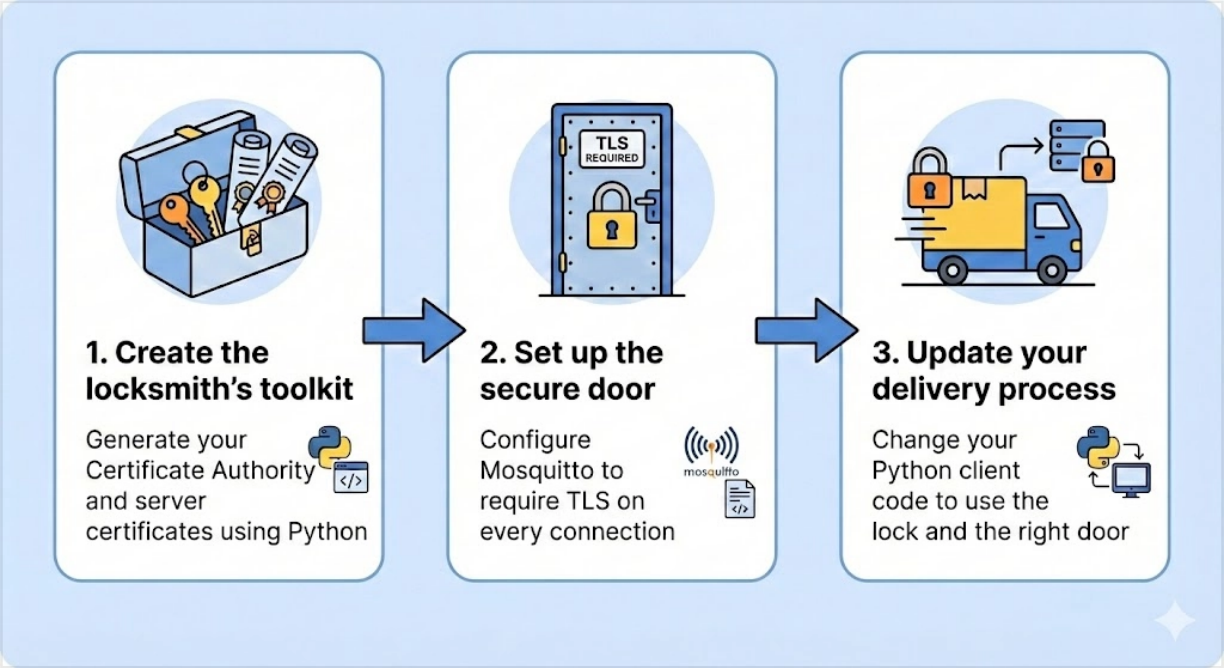
Here is a message from **Maya Chen:**
A few things to watch for — big websites use well-known CAs like DigiCert, Let's Encrypt, or Sectigo. Certificate validity varies from 90 days to 1-2 years. And you'll notice the chain of trust is visible — you can see the whole CA hierarchy.

Solution: https://docs.google.com/document/d/1tWBK0h-KxPNccMXJnG11EkpEBHoptFTrrMiCf_2XZD8/edit?usp=sharing

Step 2: Setting Up TLS

What You're About to Build

By the end of the day, anyone trying to send an unlocked suitcase (connecting without TLS) gets rejected.



Certificate Generation with Python

💡 **What's OpenSSL?** OpenSSL is a free command-line tool that's been around since 1998 — it's the Swiss Army knife of encryption. Almost every server on the internet uses it somewhere. The catch? Its commands look like someone fell

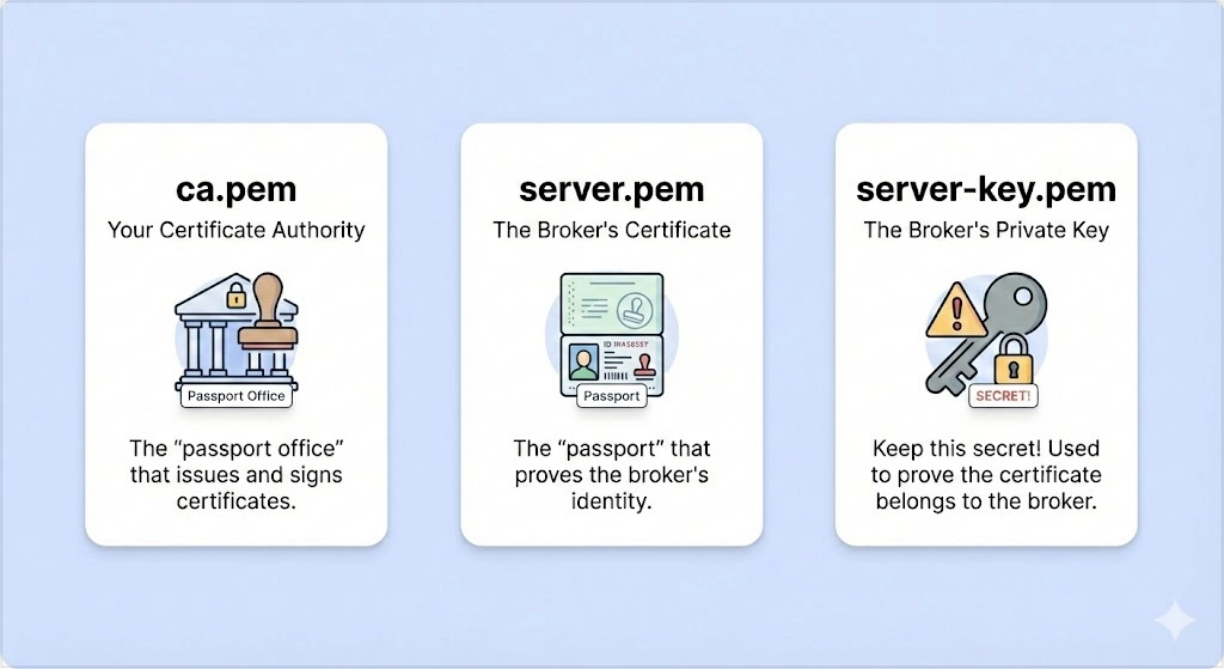
asleep on their keyboard. Something like openssl req -new -x509 -days 365 -key ca-key.pem -out ca.pem -subj "/CN=MyCA" is a perfectly normal OpenSSL command. Python's cryptography library does the same thing but in readable, debuggable code — which is why Maya prefers it for teaching.

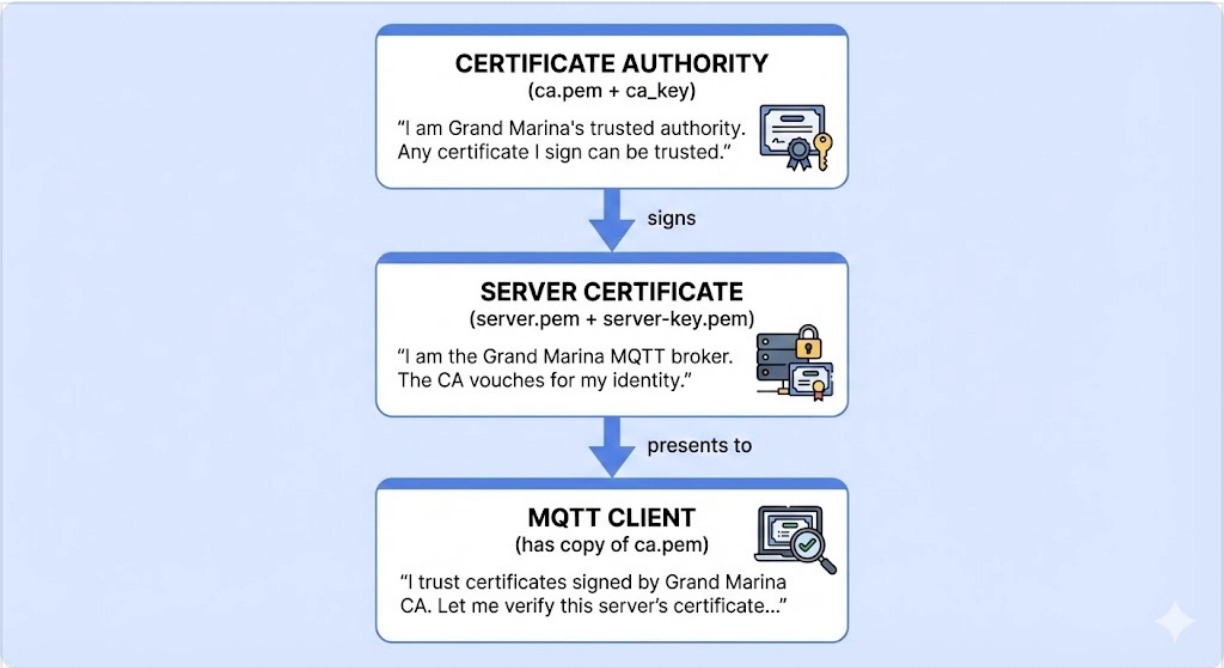
Certificate Generation

You might be thinking: "Can't I just download pre-made certificates somewhere?" Here's why generating your own matters:

1. **Understanding builds confidence.** When something goes wrong (and it will), you'll know what each file does and how to fix it.
2. **Real IoT deployments require this.** You can't use pre-made certificates for your own systems — each organization needs their own CA.

What You'll Create





The Script: generate_certs.py
This is the script that generates all three files.
Open your Python editor and create a new file called generate_certs.py

```
pip install cryptography
```

The Imports

```
import ipaddress
from datetime import datetime, timedelta, timezone
from pathlib import Path

from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes,
serialization
from cryptography.hazmat.primitives.asymmetric import rsa
```

What each import does:

Import	Purpose
ipaddress	For adding IP addresses (like 127.0.0.1) to certificates
datetime, timedelta, timezone	For setting "valid from" and "valid until" dates
Path	For file path handling
cryptography.x509	The main certificate building library

NameOID	Standard identifiers for certificate fields
hashes	Cryptographic hash algorithms (SHA256)
serialization	For saving certificates to files
rsa	RSA key generation

Part 1: Creating the Certificate Authority

Before you set up your hotel's security system, you need someone in charge of handing out the ID badges. That's what the CA does — it's the trusted authority that says "yes, this server is who it claims to be." You're building that authority first.

Before looking at the code, try to understand what the code will try to do:

Line	What It Does
.subject_name(ca_name)	Who this certificate belongs to
.issuer_name(ca_name)	Who signed it — **same as subject** because it's self-signed
.public_key(...)	Embeds the public key
.serial_number(...)	Unique ID for this certificate
.not_valid_before(...)	Certificate is valid starting now
.not_valid_after(...)	Expires in 10 years (3650 days)
.add_extension(ca=True)	**Critical:** Marks this as a CA certificate
.sign(ca_key, ...)	Signs with its own private key

That's all the code below is doing — filling out this form in Python and stamping it with a signature.

```
def generate_ca_certificate():
    """Generate the Certificate Authority (CA) certificate"""

    # Step 1: Generate a private key for the CA
    ca_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )
```

What's happening: We create an RSA key pair. The CA needs its own private key to sign other certificates.

- public_exponent=65537 — A standard, secure value (you'll see this everywhere)
- key_size=2048 — 2048-bit encryption is secure for learning; production

might use 4096

```
    # Step 2: Define the CA's identity
    ca_name = x509.Name([
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, "Grand
Marina Hotel"),
        x509.NameAttribute(NameOID.ORGANIZATIONAL_UNIT_NAME,
"Water Systems Security"),
        x509.NameAttribute(NameOID.COMMON_NAME, "Grand Marina
Root CA"),
    ])
```

What's happening: This is the "name tag" for our CA. When someone asks "Who signed this certificate?", they'll see "Grand Marina Root CA."

```
    # Step 3: Build and sign the CA certificate
    ca_cert = (
        x509.CertificateBuilder()
        .subject_name(ca_name)
        .issuer_name(ca_name)  # Self-signed: issuer = subject
        .public_key(ca_key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(datetime.now(timezone.utc))
        .not_valid_after(datetime×now(timezone×utc) +
timedelta(days=3650))
        .add_extension(
            x509.BasicConstraints(ca=True, path_length=None),
            critical=True,
        )
        .sign(ca_key, hashes.SHA256())
    )
```

💡 **Notice that issuer_name equals subject_name — the CA vouches for itself. This is normal for root CAs. It's like a government issuing its own ID: someone has to be the starting point of trust**.

Part 2: Creating the Server Certificate

The CA is set up — now it needs to actually issue an ID badge. This part creates the certificate for your MQTT broker, the one that tells every sensor: "You can trust me, the passport office vouched for me."

```
def generate_server_certificate(ca_key, ca_cert):
    """Generate the server certificate signed by the CA"""
```

```
    # The server gets its OWN key pair (separate from CA)
    server_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )
```

Important: The server has its own private key. The CA's key is only used for signing.

```
    # Server's identity
    server_name = x509.Name([
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, "Grand
Marina Hotel"),
        x509.NameAttribute(NameOID.ORGANIZATIONAL_UNIT_NAME,
"MQTT Broker"),
        x509.NameAttribute(NameOID.COMMON_NAME, "localhost"),
    ])
```

Critical: The COMMON_NAME must match the hostname clients use to connect. Since we're developing locally, we use localhost.

```
    server_cert = (
        x509.CertificateBuilder()
        .subject_name(server_name)
        .issuer_name(ca_cert.subject)  # CA is the issuer!
        .public_key(server_key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(datetime.now(timezone.utc))
        .not_valid_after(datetime×now(timezone×utc) +
timedelta(days=365))
        .add_extension(
            x509.SubjectAlternativeName([
                x509.DNSName("localhost"),

x509.IPAddress(ipaddress.IPv4Address("127.0.0.1")),
            ]),
            critical=False,
        )
        .add_extension(
            x509.BasicConstraints(ca=False, path_length=None),
            critical=True,
        )
        .sign(ca_key, hashes.SHA256())  # Signed by CA's key!
```

```
    )
```

The code looks almost identical to Part 1. Same structure, same CertificateBuilder. Spot the 4 differences:

Aspect	CA Certificate	Server Certificate
Issuer	Self (ca_name)	CA (ca_cert.subject)
Validity	10 years	1 year
ca=	True	False
Signed by	Its own key	**CA's private key**

That's it. Everything else is the same form, just filled in differently.

💡

Note: COMMON_NAME must match your hostname. Since we're developing locally, we use localhost. The SubjectAlternativeName section adds 127.0.0.1 as a backup — so your certificate works whether clients connect by name or IP address.

Subject Alternative Names (SANs)

This part is important for modern TLS:

```
x509.SubjectAlternativeName([
    x509.DNSName("localhost"),
    x509.IPAddress(ipaddress.IPv4Address("127.0.0.1")),
])
```

SANs list all the hostnames and IP addresses this certificate is valid for. Modern browsers and clients check SANs, not just the Common Name. This means your certificate works whether you connect to localhost or 127.0.0.1.

Part 3: Saving the Files

You've created the CA and the broker's certificate — but right now they only exist in memory. This part writes them out as actual files that Mosquitto can use. Think of it as printing the ID badges so people can actually carry them around.

```
def save_certificates(ca_cert, server_cert, server_key,
output_dir="certs"):
    output_path = Path(output_dir)
    output_path.mkdir(exist_ok=True)

    # Save CA certificate (public)
    with open(output_path / "ca.pem", "wb") as f:

f.write(ca_cert.public_bytes(serialization.Encoding.PEM))

    # Save server certificate (public)
```

```
    with open(output_path / "server.pem", "wb") as f:

f.write(server_cert.public_bytes(serialization.Encoding.PEM))

    # Save server private key (SECRET!)
    with open(output_path / "server-key.pem", "wb") as f:
        f.write(server_key.private_bytes(
            encoding=serialization.Encoding.PEM,

format=serialization.PrivateFormat.TraditionalOpenSSL,
            encryption_algorithm=serialization.NoEncryption()
        ))
```

What's happening:

- public_bytes(PEM) — Exports in PEM format (Base64-encoded, readable in text editors)
- private_bytes(...) — Exports the private key
- NoEncryption() — The key file isn't password-protected (for simplicity)

🚨 **Security Note:**
In production, you'd encrypt the private key with a password. We skip this for learning, but never leave private keys unprotected in real deployments.

Running the Script

```
python generate_certs.py
```

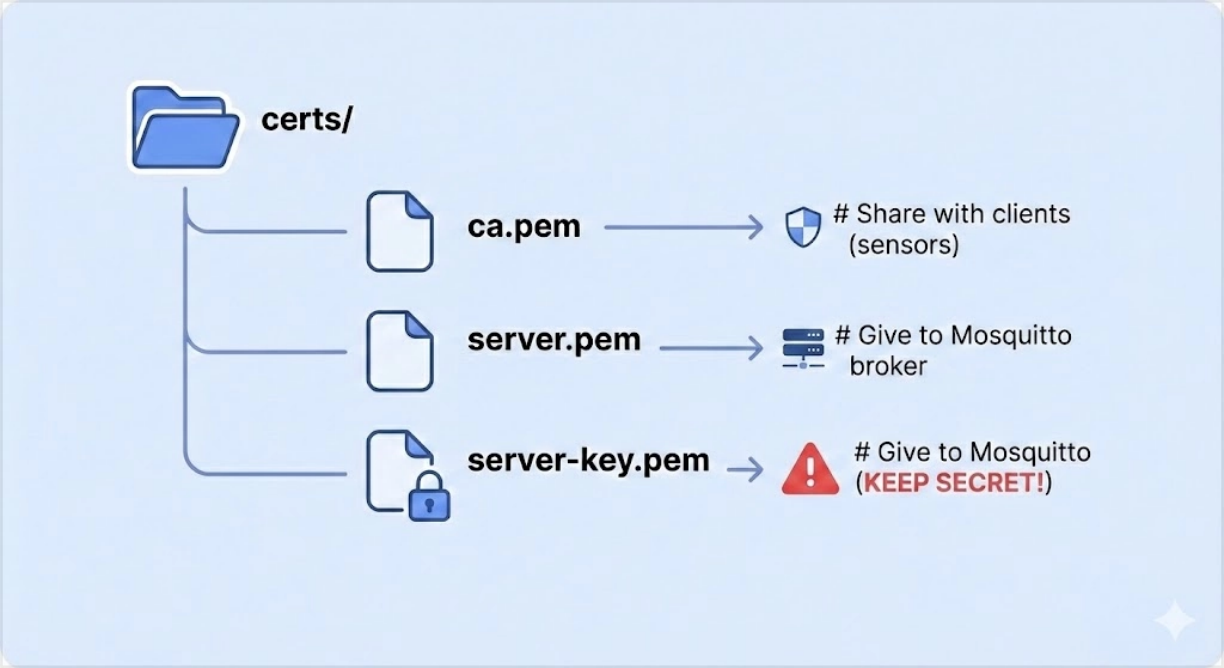
Expected output:

```
============================================================
  Certificate Generation for Grand Marina Hotel
============================================================

[1/3] Generating Certificate Authority (CA)...
      Generating CA private key (2048 bits)...
      Creating CA certificate (valid for 10 years)...
      CA certificate created successfully!

[2/3] Generating Server Certificate...
      Generating server private key (2048 bits)...
      Creating server certificate (valid for 1 year)...
      Common Name: localhost
      Subject Alternative Names: localhost, 127.0.0.1
      Server certificate created successfully!
```

```
[3/3] Saving certificates to certs/ folder...
      Saved: certs\ca.pem
      Saved: certs\server.pem
      Saved: certs\server-key.pem

      Verifying certificates...
      CA Subject: CN=Grand Marina Root CA,OU=Water Systems
Security,O=Grand Marina Hotel
      CA Valid Until: 2036-02-11
      Server Subject: CN=localhost,OU=MQTT Broker,O=Grand
Marina Hotel
      Server Issuer: CN=Grand Marina Root CA,OU=Water Systems
Security,O=Grand Marina Hotel
      Server Valid Until: 2027-02-13
      Chain verified: Server cert is signed by CA

============================================================
  Certificates generated successfully!
============================================================

Files created:
  certs/ca.pem         - CA certificate (share with clients)
  certs/server.pem     - Server certificate (for Mosquitto)
  certs/server-key.pem - Server private key (keep secret!)
```

What You Created

After running the script, you have:



You just created the trust infrastructure for Grand Marina's water monitoring system:

1. **CA certificate** — Your sensors will trust any certificate signed by this CA
2. **Server certificate** — Proves the broker is legitimate
3. **Server private key** — Lets the broker prove it owns the certificate

Next up, you'll configure Mosquitto to use these files. Then clients will be able to establish secure, encrypted connections.

Your Current Config (Insecure)

```
# mosquitto_insecure.conf
listener 1883
bind_address localhost
allow_anonymous true
log_type all
log_dest stdout
```

What this means:

Line	Meaning
listener 1883	Accept connections on port 1883
bind_address localhost	Only accept local connections
allow_anonymous true	No username/password required
log_type all	Log everything

log_dest stdout	Print logs to terminal

This works, but offers **zero encryption**. Anyone on the network can read your messages.

The TLS Config

```
# mosquitto_tls.conf
listener 8883
bind_address localhost

# TLS Certificate Configuration
cafile certs/ca.pem
certfile certs/server.pem
keyfile certs/server-key.pem

# Client certificate requirement
require_certificate false

# Authentication
allow_anonymous true

# Logging
log_type all
log_dest stdout
```

Change 1: Port Number

```
listener 8883
```

The biggest tell is the port. When you see **8883**, you know **TLS** is expected. It's the industry standard — same idea as HTTP on port 80 versus HTTPS on port 443.

Change 2: Certificate Paths

```
cafile certs/ca.pem
certfile certs/server.pem
keyfile certs/server-key.pem
```

These three lines tell Mosquitto where your certificates live. That's why we generated them first.

Setting	File	Purpose
cafile	ca.pem	The CA certificate — used to verify client certificates (in mTLS)
certfile	server.pem	The broker's certificate — sent to clients

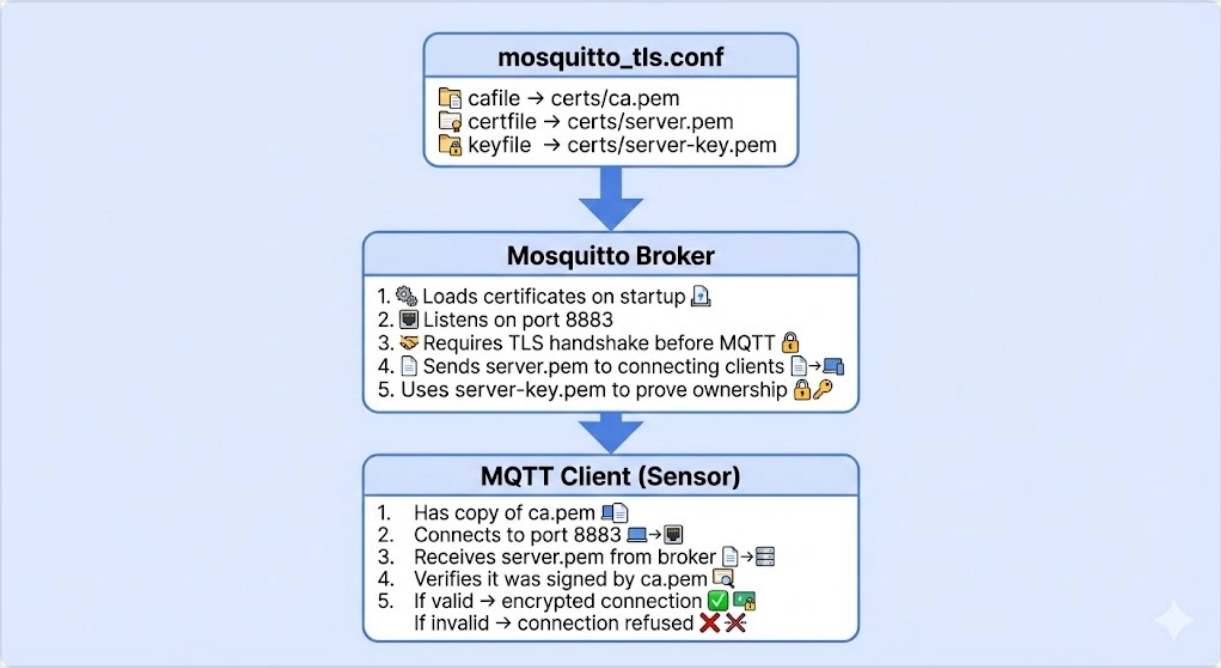
keyfile	server-key.pem	The broker's private key — proves ownership of certificate

💡 **Note: Path Gotcha**
These paths are **relative to where you run Mosquitto from.** If you run `mosquitto -c mosquitto_tls.conf` from `/home/user/project4/`, then Mosquitto looks for `/home/user/project4/certs/ca.pem`.

Change 3: Client Certificate Requirement
`require_certificate false`
It controls whether sensors need their own certificates to connect. For now, we set it to false — the broker proves its identity, but the sensors don't need to prove theirs. Same way most websites work. You visit Amazon, your browser checks Amazon's certificate, but Amazon doesn't ask your browser for one.

Setting	Meaning
require_certificate false	Clients don't need certificates (one-way TLS)
require_certificate true	Clients MUST have certificates (mutual TLS / mTLS)

How the Files Connect



Starting Mosquitto with TLS
```
mosquitto -c mosquitto_tls.conf -v
```
Expected output (success):
```
1705632000: mosquitto version 2.0.18 starting
1705632000: Config loaded from mosquitto_tls.conf.
```

```
1705632000: Opening TLS listener on port 8883.
1705632000: mosquitto version 2.0.18 running
```

Verifying TLS is Working

Now try connecting without TLS — just to prove the broker rejects it.

```
mosquitto_sub -h localhost -p 8883 -t "test/#"
```

Expected result: Connection fails with Error: A TLS error occurred.

That error is a good thing. It means the broker is doing its job — no encryption, no entry. If something isn't working, check what's actually listening on which ports:

On Windows:

```
netstat -an | findstr "8883"
```

On Mac/Linux:

```
lsof -i :8883
```

The broker is locked down. But your Python code is still trying to knock on the old door — port 1883, no encryption. Time to update your clients.

Securing Mosquitto with TLS Certificates

Your Current Config (Insecure)

```
# mosquitto_insecure.conf
listener 1883
bind_address localhost
allow_anonymous true
log_type all
log_dest stdout
```

What this means:

Line	Meaning
listener 1883	Accept connections on port 1883
bind_address localhost	Only accept local connections
allow_anonymous true	No username/password required
log_type all	Log everything
log_dest stdout	Print logs to terminal

This works, but offers **zero encryption**. Anyone on the network can read your messages.

The TLS Config

```
# mosquitto_tls.conf
listener 8883
bind_address localhost

# TLS Certificate Configuration
```

```
cafile certs/ca.pem
certfile certs/server.pem
keyfile certs/server-key.pem

# Client certificate requirement
require_certificate false

# Authentication
allow_anonymous true

# Logging
log_type all
log_dest stdout
```

Change 1: Port Number

```
listener 8883
```

The biggest tell is the port. When you see **8883**, you know **TLS** is expected. It's the industry standard — same idea as HTTP on port 80 versus HTTPS on port 443.

Change 2: Certificate Paths

```
cafile certs/ca.pem
certfile certs/server.pem
keyfile certs/server-key.pem
```

These three lines tell Mosquitto where your certificates live. That's why we generated them first.

Setting	File	Purpose
cafile	ca.pem	The CA certificate — used to verify client certificates (in mTLS)
certfile	server.pem	The broker's certificate — sent to clients
keyfile	server-key.pem	The broker's private key — proves ownership of certificate

💡 **Note: Path Gotcha**

These paths are **relative to where you run Mosquitto from.** If you run `mosquitto -c mosquitto_tls.conf` from `/home/user/project4/`, then Mosquitto looks for `/home/user/project4/certs/ca.pem`.

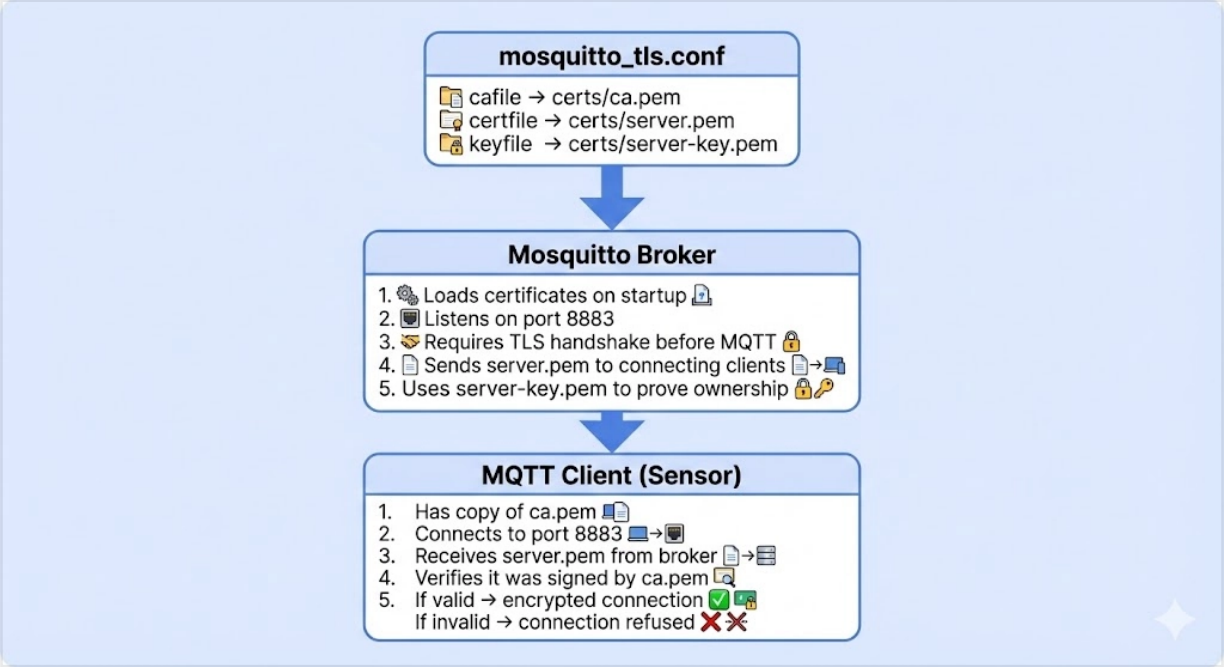
Change 3: Client Certificate Requirement

```
require_certificate false
```

It controls whether sensors need their own certificates to connect. For now, we set it to false — the broker proves its identity, but the sensors don't need to prove theirs. Same way most websites work. You visit Amazon, your browser checks Amazon's certificate, but Amazon doesn't ask your browser for one.

Setting	Meaning
require_certificate false	Clients don't need certificates (one-way TLS)
require_certificate true	Clients MUST have certificates (mutual TLS / mTLS)

How the Files Connect



Starting Mosquitto with TLS

```
mosquitto -c mosquitto_tls.conf -v
```

Expected output (success):

```
1705632000: mosquitto version 2.0.18 starting
1705632000: Config loaded from mosquitto_tls.conf.
1705632000: Opening TLS listener on port 8883.
1705632000: mosquitto version 2.0.18 running
```

Verifying TLS is Working

Now try connecting without TLS — just to prove the broker rejects it.

```
mosquitto_sub -h localhost -p 8883 -t "test/#"
```

Expected result: Connection fails with Error: A TLS error occurred.

That error is a good thing. It means the broker is doing its job — no encryption, no entry. If something isn't working, check what's actually listening on which ports:

On Windows:

```
netstat -an | findstr "8883"
```
On Mac/Linux:
```
lsof -i :8883
```
The broker is locked down. But your Python code is still trying to knock on the old door — port 1883, no encryption. Time to update your clients.

TLS Python Client Code

The Problem: Your Code Still Speaks Plaintext
Your Week 3 publisher code connects like this:
```
client.connect("localhost", 1883, keepalive=60)
```
Port 1883. No encryption. The broker won't let this through the door anymore — it's TLS-only on 8883 now. You need to update your Python clients to speak TLS. Here are the three changes you'll need to make:

Change	What	Why
1	Import ssl module	Python needs this library to understand TLS protocols and encryption standards
2	Add TLS configuration	Tell your client where to find the CA certificate so it can verify the broker's identity
3	Call client.tls_set()	Activate TLS on the connection — without this call, your client still connects in plaintext

Change 1: Import the ssl Module in Your Publisher/Subscriber Script
Remember the publisher and subscriber scripts you built in Week 3? Those are the files you'll be updating. At the top of either script, add one import:
```
import ssl   # ADD THIS FOR TLS
```
This gives you access to SSL/TLS constants like ssl.CERT_REQUIRED and ssl.PROTOCOL_TLS.

Change 2: Add TLS Configuration
Instead of scattering values like "certs/ca.pem" and 8883 throughout your code, group them into a config dictionary. This way, if you ever need to change the certificate path or broker address, you update one place instead of hunting through every file:
```
# ============================================
# TLS CONFIGURATION - ADD THIS FOR TLS
# ============================================
TLS_CONFIG = {
```

```
    "ca_certs": "certs/ca.pem",      # Path to CA certificate
    "broker_host": "localhost",
    "broker_port": 8883,              # TLS port (not 1883!)
}
# ==============================================
```

What each setting does:

Setting	Value	Purpose
ca_certs	certs/ca.pem	Path to CA certificate for verification
broker_host	localhost	Broker hostname
broker_port	8883	TLS port (not 1883)

💡 **Why Only the CA Certificate?**

Clients only need ca.pem — the CA certificate. They don't need the server's private key (that would defeat the purpose!). The client uses the CA certificate to verify that the broker's certificate is legitimate.

Change 3: Configure TLS on the Client

Having the config dictionary isn't enough — your client doesn't know it should use TLS until you explicitly tell it. The client.tls_set() call is what actually activates encryption. Add this before your client.connect() line:

```
# ==============================================
# CONFIGURE TLS - ADD THIS FOR TLS
# ==============================================
client.tls_set(
    ca_certs=TLS_CONFIG["ca_certs"],    # Trust this CA
    certfile=None,                       # No client cert
(server-only TLS)
    keyfile=None,                        # No client key
    cert_reqs=ssl.CERT_REQUIRED,         # Verify server
certificate
    tls_version=ssl.PROTOCOL_TLS,        # Use modern TLS
)
# ==============================================
```

What each parameter does:

Parameter	Value	Purpose
ca_certs	Path to CA certificate	Client trusts certs signed by this CA
certfile	None	Client certificate (not needed for one-way TLS)

keyfile	None	Client private key (not needed for one-way TLS)
cert_reqs	ssl.CERT_REQUIRED	**Must** verify the server's certificate
tls_version	ssl.PROTOCOL_TLS	Use modern TLS (1.2 or 1.3)

💡 **Why CERT_REQUIRED?**
Emphasising again — setting cert_reqs=ssl.CERT_REQUIRED is crucial. It means the client will verify: (1) the server has a valid certificate, (2) it was signed by our trusted CA (ca.pem), (3) it hasn't expired, and (4) the hostname matches. If **any** of these checks fail, the connection is refused. This prevents Person-in-the-Middle attacks.

Why No Client Certificate?
You might notice we set certfile=None and keyfile=None.
For this week, we will use **one-way TLS** (also called server-only TLS):

- The server proves its identity to clients via certificate
- Clients don't prove their identity via certificate

This is how most websites work — you verify the bank is real, but you don't need a certificate to visit the site.

The Connection Call
You've imported ssl, set up your config, and called tls_set(). The last piece is the connection itself — and it's the smallest change. Your connection call only changes the port:

```
# Before (Project 3):
client.connect("localhost", 1883, keepalive=60)

# After (Project 4):
client.connect(
    TLS_CONFIG["broker_host"],
    TLS_CONFIG["broker_port"],       # Port 8883, not 1883!
    keepalive=60
)
```

Best Practice (Important): Check Certificate Existence
Before attempting to connect, verify that the CA certificate exists. This prevents confusing SSL errors when certificates are missing:

```
from pathlib import Path

ca_path = Path(TLS_CONFIG["ca_certs"])
if not ca_path.exists():
    print(f"CA certificate not found: {ca_path}")
```

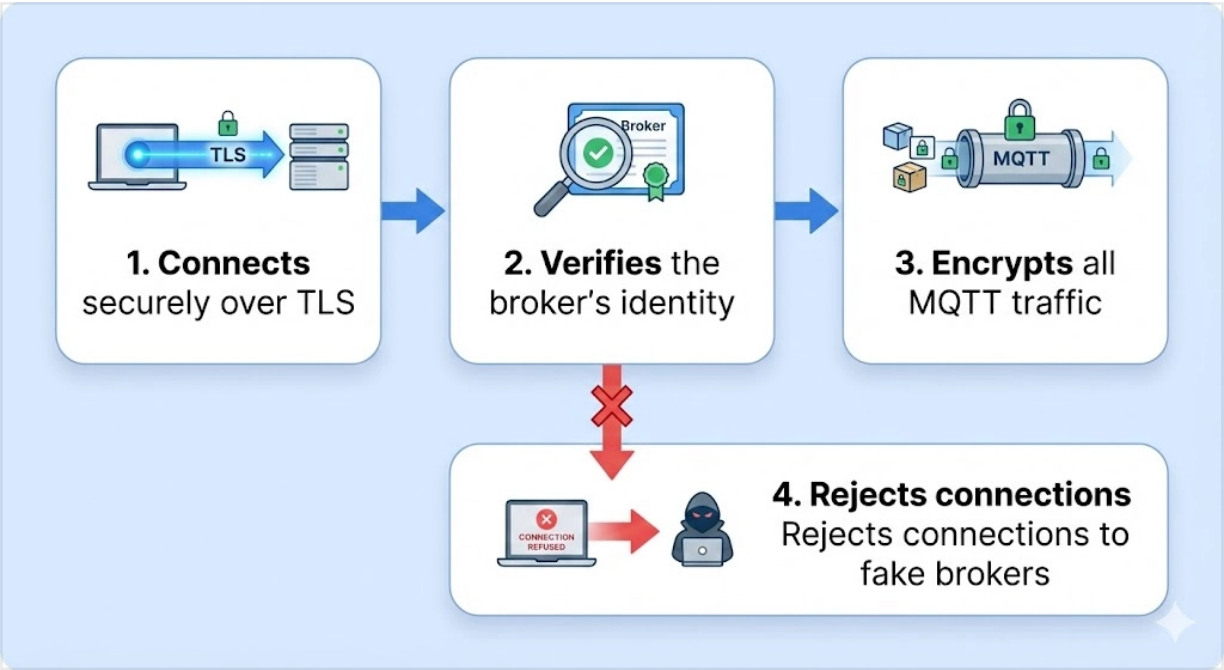
```
    print("Run generate_certs.py first!")
    return  # Exit early
```

How It All Connects

```
1. Client starts connection to localhost: 8883
                    │
                    ▼
2. TLS handshake begins
   - Client says "Hello, I support TLS 1.2/1.3"
   - Broker sends server.pem certificate
                    │
                    ▼
3. Client verifies certificate using ca.pem
   - Is it signed by our CA? ✓
   - Is it expired? ✓
   - Does hostname match? ✓
                    │
                    ▼
4. If all checks pass → Encrypted channel established
   If any check fails → Connection refused
                    │
                    ▼
5. MQTT messages flow through encrypted channel
   - Publish, subscribe, etc. work normally
   - All data is encrypted
```

What You Will Accomplish

After making these three changes, your client:



Task: Secure Your Pipeline (Mandatory)

What You'll Do in This Task

1. Generate certificates using our Python script
2. Configure Mosquitto to require TLS
3. Update your publisher and subscriber
4. Verify insecure connections are rejected

Follow the Steps Below:

Step 1: Set Up Your Project Folder

Now open a terminal and navigate to your new project-04 folder. You'll run all commands from here.

Note: Keep this terminal open. You'll need multiple terminal windows for the steps ahead. Every time you open a new terminal, you'll need to cd into the project-04 folder again.

Step 2: Generate Your Certificates

In the terminal you just opened (which should be in your project-04 folder), run:

```
python generate_certs.py
```

Verify the files exist:

```
ls certs/
You should see: ca.pem, server.pem, server-key.pem
```

Step 3: Stop Any Running Mosquitto

Before starting with TLS, stop any existing instances:

```
pkill mosquitto
```

Step 4: Start Mosquitto with TLS

In the same terminal (still in your project-04 folder), start Mosquitto with the TLS config:

```
mosquitto -c mosquitto_tls.conf -v
```

Step 5: Test That Insecure Connections Fail

Before running your secure clients, verify that insecure connections are rejected. In a new terminal:

```
mosquitto_sub -h localhost -p 8883 -t "test/#"
```

Step 6: Run Your TLS-Enabled Subscriber

In the same terminal from Step 5 (the insecure test is done — that terminal is free now), make sure you're still in the project-04 folder and run:

```
python subscriber_tls.py
```

Step 7: Run Your TLS-Enabled Publisher

Open a **third** terminal window, cd into your project-04 folder again, and run:

```
python publisher_tls.py
```

Step 8: Deliverable

Once you are done, upload a screenshot at the end of this task module showing:

1. Your subscriber terminal receiving TLS-encrypted messages
2. The Mosquitto broker showing "Opening TLS listener on port 8883"